real-tr-p16-full

An AgentMPI run analysis

Abstract

This is the $p = 16$ point of the parallel book-translation strong-scaling series, and the only outright failure in the 500-run archive: `job.finish` records `ok: false` with all sixteen ranks in `failed_ranks`, no translated section on disk, and $0.0726 spent for nothing. The trace shows a single cause with a long shadow. Six of the sixteen rank roles were never bound to a worker process at all — the fabric's `ranks` table still lists ranks 9–13 and 15 with `executor = broker` at incarnation 1, and their six queued term-extraction calls carry `claimed_at = NULL` for the whole run — so the glossary `allreduce` could never complete, and the ten ranks that did their work correctly died waiting for the six that had no executor. Every claim below traces to this run's event log under `traces/events/` or to its fabric under `runs/`. The reading is that this is a capacity failure *observed through* AgentMPI rather than a failure *of* it: the protocol detected the condition, named it with the right error class, bounded every wait, and refused to report success. The one genuine defect is a composition defect — the broker's 5,400 s expiry is three times the collectives' 1,800 s receive timeout, so the harness's degrade-and-stay-in-the-group discipline fires 3,600 s after the peers it was meant to rescue have already given up. Because those two constants are independently authored defaults in different modules and no line of the harness is wrong, §7.7 argues that AgentMPI should own that ordering — reporting it, not forbidding it — while the mitigation stays the harness author's responsibility. The single most consequential thing a reader should take away is nonetheless external to the run: this failed run was a row in the paper's strong-scaling table, and the p=16 point is not safe to use, for two independent reasons.

1 What this run is

property	value
run	`real-tr-p16-full`
experiment	`translation`
campaign label	`real-tr`
declared world size	16
ranks appearing in the log	16
executors	`broker` × 16, `worker` × 20
events	207
wall time	7,200.21 s
job outcome	**failed**

2 Headline measurements

metric	value	meaning
achieved parallelism	0.04×	total busy time over wall time
parallel efficiency	0.3%	achieved parallelism per rank
idle fraction	99.7%	rank-seconds paid for and unused
peak ranks busy	8	of 16 in the world
serial fraction of active time	16.7%	time with exactly one rank busy
load imbalance	1.21×	slowest rank's busy time over the mean
coordination share	0.0%	rank-seconds blocked in collectives, of those available
coordination in flight	0.1%	wall clock with any rank inside a collective
rank reattachments	20	fresh agent processes that took over a rank role
agent calls	10	invocations that reached a model
agent latency p50 / p95	126.57 / 184.56 s	per invocation
messages	27	24 eager, 3 rendezvous
tokens sent	34,525	16,082 deferred under rendezvous
tokens in / out	11,356 / 2,569	prompt and completion
cost	$0.0726	0.0283 $/kTok out

3 What the analysis notices

- [critical] The job recorded failure: 16 of 16 ranks were marked failed.
- [critical] At least one rank waited 5,400 s for an agent to claim its task before the broker gave up. That is a pool-sizing failure outside the protocol, not a slow run.
- [warning] `reduce/binomial` (label `glossary`) was reached by 12 of 16 ranks, so it never completed.
- [note] Parallel efficiency is 0.3%: 99.7% of the rank-seconds paid for went unused.
- [warning] The coordination figures count time inside *completed* collectives only, and this run has ranks that never completed one. A rank records a collective on completion, so a rank that blocked and then timed out contributes nothing — the reported share is a floor, and on a badly degraded run a very loose one.
- [ok] 20 rank reattachment(s) occurred, up to incarnation 3: agent processes were replaced mid-run while the rank roles, their mailboxes, and their context accounts persisted.
- [ok] All 1 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.

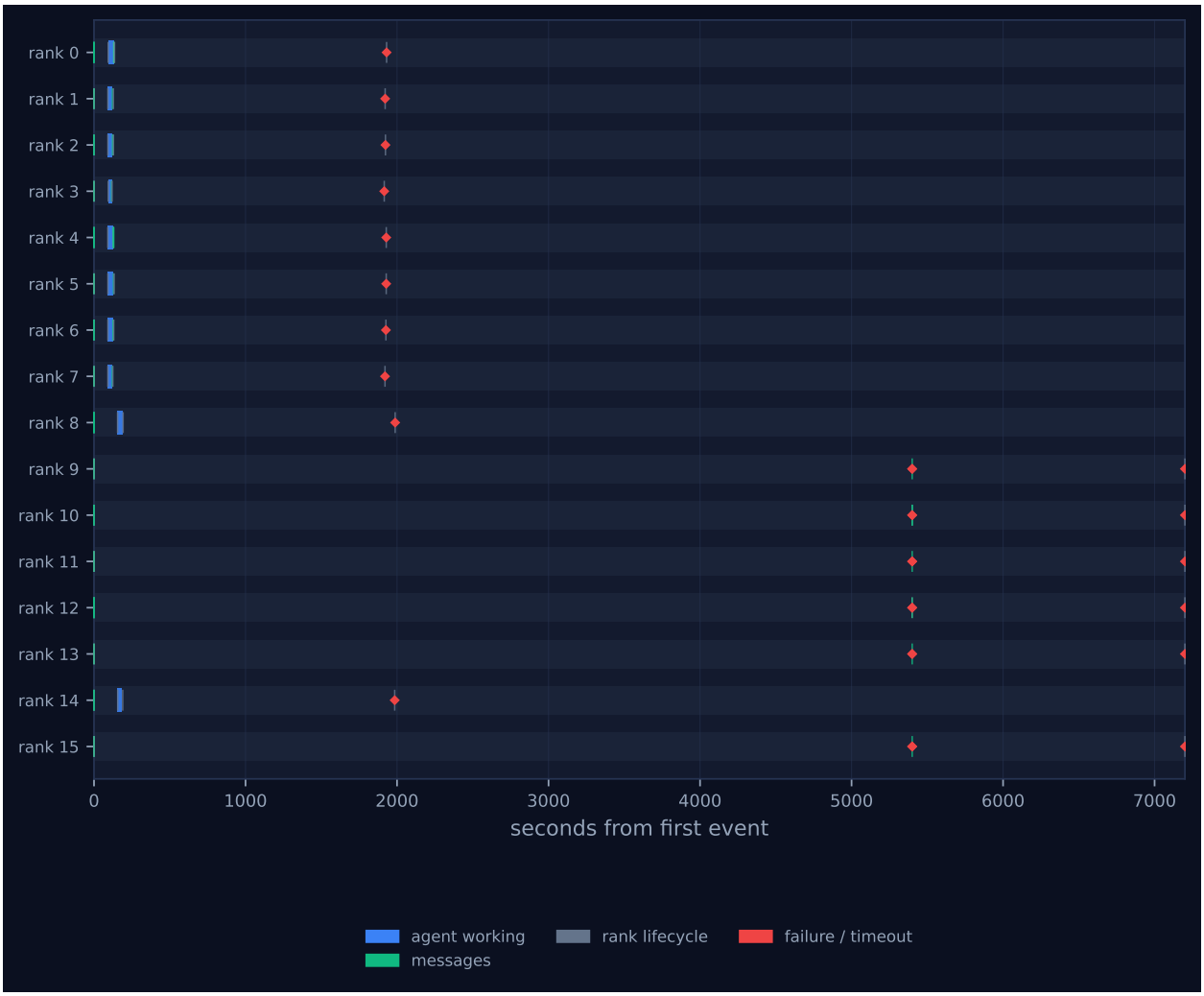

Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

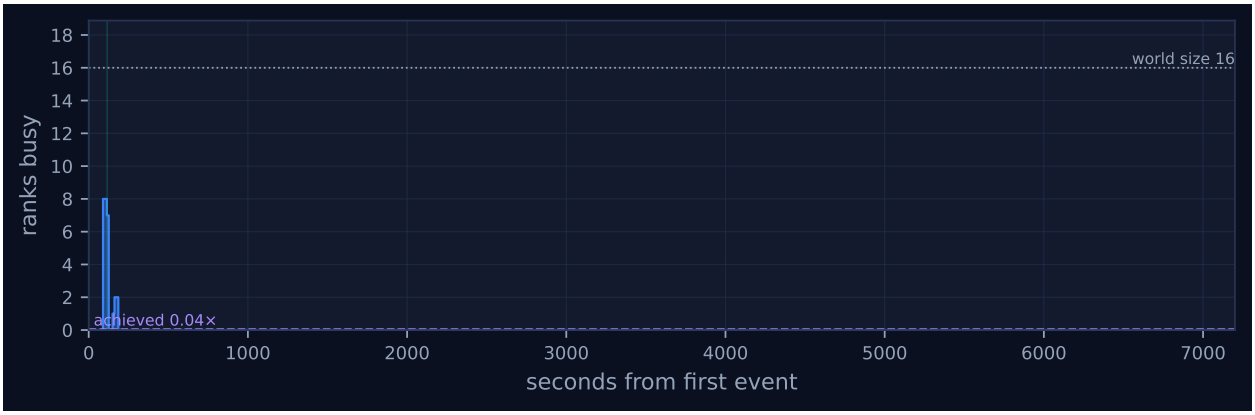

Figure 2: Ranks simultaneously busy over time. The dotted line is the declared world size and the dashed line the achieved parallelism; the gap between them is what the run paid for and did not use. Faint vertical lines mark collective invocations.

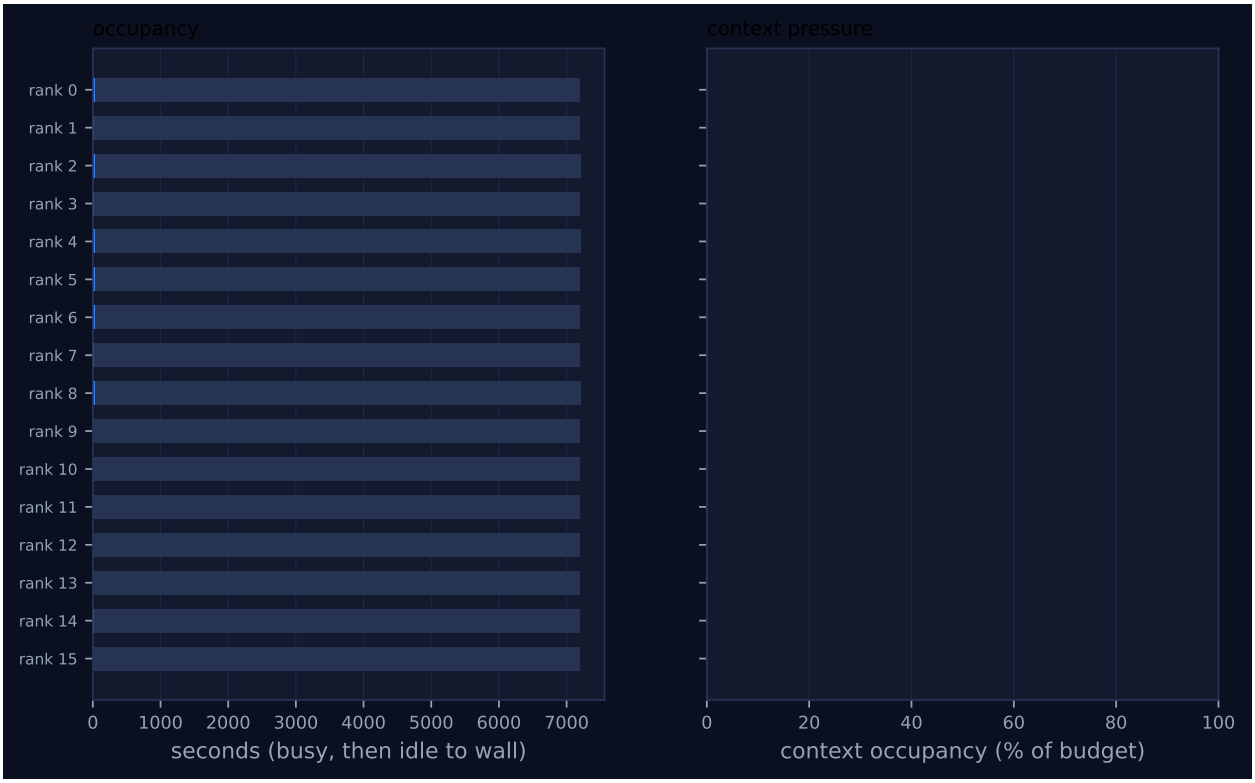

Figure 3: Per-rank occupancy and context pressure. Left: busy time, with the remainder to wall time in grey. Right: context occupancy against budget, amber above half and red above 80%, where admission pressure starts to reject artefacts.

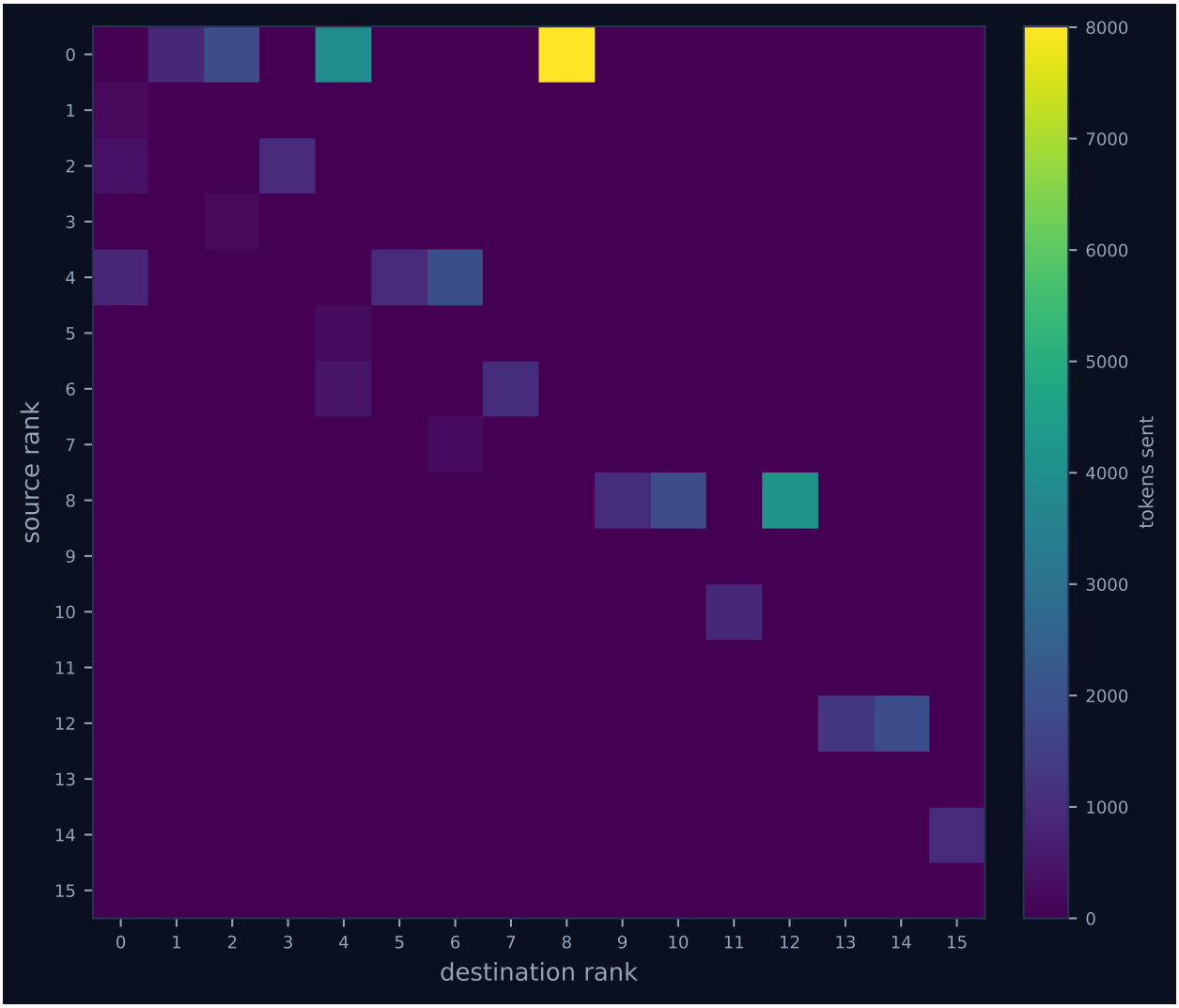

Figure 4: Communication matrix: tokens sent from each rank to each rank. The algorithm's shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

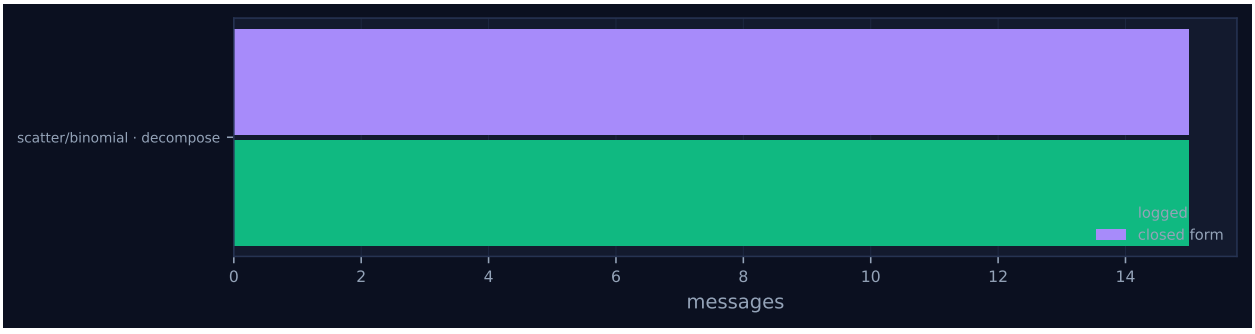

Figure 5: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	37.50	1.9	1	1	4	3	14,744	0.0	failed
1	29.59	1.5	1	1	1	1	208	0.0	failed
2	31.37	1.6	1	1	2	2	1,334	0.0	failed
3	21.62	1.1	1	1	1	1	195	0.0	failed
4	31.75	1.6	1	1	3	3	3,791	0.0	failed
5	36.23	1.9	1	1	1	1	253	0.0	failed
6	34.28	1.8	1	1	2	2	1,518	0.0	failed
7	28.60	1.5	1	1	1	1	242	0.0	failed
8	31.75	1.6	1	1	3	1	7,055	0.0	failed
9	0.00	0.0	0	0	1	1	21	0.0	failed
10	0.00	0.0	0	0	2	2	958	0.0	failed
11	0.00	0.0	0	0	1	1	21	0.0	failed
12	0.00	0.0	0	0	2	2	3,184	0.0	failed
13	0.00	0.0	0	0	1	1	21	0.0	failed
14	27.80	1.4	1	1	1	1	959	0.0	failed
15	0.00	0.0	0	0	1	1	21	0.0	failed

Per-rank behaviour. Occupancy is busy time over the rank's own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)	skew (s)	model
scatter	binomial	decompose	16	16	4	4	15	15	31,679	0.08	0.06	✓
reduce	binomial	glossary	16	12	4	4	12	15	2,509	5.74	5,284.58	–

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

6 Reading the timeline

Figure 1 is almost entirely empty, and the emptiness is the finding. The axis runs to 7,200 s. Every blue bar in the figure — every interval in which a rank held an agent task — lies inside the first 188 s, or 2.6% of the axis. The union of those intervals is 72.0 s, one per cent of the wall clock, against a declared world of sixteen ranks. The remaining 7,012 s contain four coloured marks per lane and nothing else.

The blue work arrives in two batches, not one, and the split is the whole story. Eight lanes (ranks 0–7) show a bar beginning at $t \approx 91$–94 s; two more (ranks 8 and 14) show a bar beginning at

$t \approx 156$ s; and six lanes (ranks 9, 10, 11, 12, 13, 15) show *no blue at all*. Those six lanes are the visual signature of a rank that never received an executor, and they are visually distinct from a rank that worked and then failed. Figure 2 renders the same fact as a step function: concurrency rises to exactly 8, falls to 0, rises to 2, and then flatlines at zero for 7,013 s beneath a dotted world-size line at 16. The achieved parallelism annotation reads 0.04×.

The gap between $t \approx 0.09$ s and $t \approx 91$ s is provisioning latency, not computation. All sixteen `broker.enqueue` events are complete by $t = 0.09$ s: the `scatterv` finished in 0.08 s with 15 messages, exactly the closed form for binomial scatter at $p = 16$, and every rank immediately published its term-extraction prompt to the fabric. Nothing then happens for ninety seconds, because a queued call can only be claimed by a worker process bound to that same rank — the broker's queue is per-rank by design, so that a rank's accumulated context cannot be stolen by another role. The first ten `rank.init` events with `executor: worker` are the external worker fleet coming up, and they are what unblocks the eight-then-two pattern. Once a worker did attach, the work itself was quick and unremarkable: subtracting `broker.claim` from `broker.complete` gives per-call service times of 21.6–37.5 s, against 19.5–29.5 s for the same prompts in `real-tr-p8-full` — a fair comparison because that run's `max_claim_wait_s` is 0.0, so its recorded latencies are service times too. The agents were not slow. Only ten of them existed.

The red marks fall in three temporally separated columns, and reading them left to right gives the causal order directly. The first column, at $t = 1{,}915$–$1{,}988$ s, carries ten `rank.error` events, one for each rank that had completed a term sheet: ranks 3, 7, 1, 2, 6, 5, 4, 0 between 1,915.60 s and 1,931.11 s, then ranks 14 and 8 at 1,984.65 s and 1,987.65 s. The trailing position of the last two is their late worker attachment carried forward unchanged. Every one of these ten died exactly $1{,}800.0 \pm 0.1$ s after its own `agent.call` completed — rank 3 at $115.58 \to 1{,}915.60$, rank 0 at $131.08 \to 1{,}931.11$, rank 8 at $187.60 \to 1{,}987.65$ — which is the default receive timeout of `algorithms.reduce`. These ranks were blocked inside the glossary reduce.

The second column, at $t = 5{,}400.15$ s, is the six unprovisioned ranks: six `broker.expire` events with `waited_s: 5400.0`, each immediately followed by `harness.degraded` with `error_class: ERR_TIMEOUT` and the label of the task that never ran (`terms:u9`, `terms:u10`, `terms:u11`, `terms:u12`, `terms:u13`, `terms:u15`). This is `safe_agent` in `experiments/translation.py` behaving exactly as its docstring promises: the local failure did not remove the rank from the group. Five of the six went straight on to contribute an empty term sheet to the reduce, and the 21-token `msg.send` events with tag `_ampi:reduce:0:3` at $t = 5{,}400.15$ s are those contributions. They arrived 3,412 s too late. Ranks 9 and 10 sent to rank 8, which had been dead since $t = 1{,}987.65$ s; rank 15 sent to rank 14, dead since $t = 1{,}984.65$ s. Those three sends (mid 23, 25 and 27) are the only unmatched messages in the run, and they are why the log holds 27 `msg.send` against 24 `msg.recv`.

The third column, at $t = 7{,}200.20$–$7{,}200.21$ s, is the remaining six ranks timing out. Again the interval is 1,800.05 s from the moment each entered the reduce at $t = 5{,}400.15$ s. It is worth being precise about this number, because 7,200 s looks like a cap and is not one: this run was launched with `job_timeout: 14400.0`, so it had another two hours of budget it never used. All sixteen `rank.error` records carry the identical payload `AmpiTimeout('receive timed out')` / `ERR_TIMEOUT`; not one is a job abort. The run's total duration is a derived quantity, $5{,}400 + 1{,}800$, and a longer job budget would have changed nothing.

Figure 4 shows the consequence in the communication matrix. The binomial scatter's outbound fan from rank 0 and the tree's internal forwarding are all there, but the reduce's returning traffic is a fragment: rank 0 received 208, 382 and 872 tokens from ranks 1, 2 and 4, and nothing from rank 8. Reconstructing the tree from the message log gives $0 \leftarrow \{1, 2, 4, 8\}$, $2 \leftarrow \{3\}$, $4 \leftarrow \{5, 6\}$, $6 \leftarrow \{7\}$,

$8 \leftarrow \{9, 10, 12\}$, $10 \leftarrow \{11\}$, $12 \leftarrow \{13, 14\}$, $14 \leftarrow \{15\}$. Rank 8's three children are ranks 9, 10 and 12 — all three unprovisioned. So rank 8 could not fold, so the root could not complete, so the entire left half of the tree, which had done its work correctly and on time, failed too. Figure 3 makes the same partition arithmetically: ten ranks at 21.6–37.5 s busy, six at exactly 0.00 s, all sixteen in state `failed`.

Figure 5 plots one bar pair, not two. The scatter's 15 logged messages sit exactly on its closed-form 15. The reduce is absent from the figure entirely: the tooling is declining to plot a comparison it has judged not to be meaningful, for the reason set out in §7.8.

7 Interpretation

7.1 What is true by construction

The harness, the decomposition and the collectives are all as `experiments/translation.py` asked for them. A binomial `scatterv` of sixteen units, a per-rank contract-checked term-sheet extraction, a `reduce_bcast` `allreduce` under `UNION` to agree the glossary, and a `PROCEED` barrier before the `gather`: the phase structure in the log is the phase structure in the source. The executor was `broker`, so the ten agent calls that completed were served by real agents and not by a surrogate — this run does tell us something about agent behaviour, although as it turns out very little, because the run died in phase 1 of five. Sixteen agent calls were enqueued in total, ten completed, and the run's `output/` directory contains zero files. No section of the book was ever translated.

Two headline numbers are also true by construction rather than emergent. The scatter's message count agreeing with its closed form is a real check but a trivially passed one at $t < 0.1$ s. And the 20 rank reattachments to incarnation 3 are the worker protocol working normally: each `ampi worker next` invocation registers a fresh `RankRuntime` before claiming, so ten served ranks × two invocations gives exactly 20. The finding that these reattachments preserved rank identity, mailboxes and context accounts is correct and is *[ok]*, but it is a property of the ten ranks that had workers. The six that did not are frozen at incarnation 1 for 7,200 s, which is the more informative half of the same measurement.

7.2 The cause: a provisioning shortfall, not congestion

The proximate cause is narrower and more mundane than the trace's shape suggests, and four independent records in the fabric agree on it. First, the `ranks` table lists ranks 0–8 and 14 with `executor = 'worker'` at incarnation 3, and ranks 9–13 and 15 with `executor = 'broker'` at incarnation 1: the six were never bound to a worker process. Second, the `agent_calls` table shows aids 4, 8, 9, 11, 14 and 16 in state `expired` with `claimed_at = NULL` — never claimed, as opposed to claimed and abandoned. Third, the event log contains zero `broker.reclaim` events, so no worker ever took one of those tasks and dropped it. Fourth, the log contains exactly 20 `rank.init` events with `executor: worker`, spread over those same ten ranks and no others.

This distinction matters because "a shortage of available agents in the shared worker pool" is not quite what happened. AgentMPI's broker queue is deliberately *not* a shared pool: `executor.claim_next` selects `WHERE rank=? AND state='pending'`, and the docstring is explicit that a worker cannot steal work belonging to another rank's role, precisely so that a rank's identity and accumulated context survive across incarnations. A worker bound to rank 4 therefore *cannot* help rank 9, however idle it is. This is visible in the trace: all eight first-wave workers had finished by $t = 131.02$ s, and

if the fleet had been a recycling pool of eight it would have drained the remaining six tasks by $t \approx 170$ s. Instead two more workers attached at $t = 155.6$ and 156.4 s and then no worker ever attached again for 7,013 s. Ten rank roles were served; six were never served at all.

So the failure is a capacity problem, and specifically a provisioning problem in the run driver rather than contention inside the protocol. It also means the shortfall is not merely *first* in the causal order — it is a precondition of the run, already true at $t = 0$ before any protocol event was recorded.

7.3 The causal order, settled

The timestamps admit only one ordering. The starvation begins at $t = 0.09$ s, when six calls enter a queue no worker will ever poll. The receive timeouts occur at $t = 1{,}915$–$1{,}988$ s. The broker expiries occur at $t = 5{,}400.15$ s. The receive timeouts are therefore downstream of the starvation by 1,800 s and *upstream* of the broker expiries by 3,400 s.

The protocol-problem reading — that receive timeouts somehow starved the claims — is excluded by more than ordering. It requires a mechanism by which a blocked receive prevents a claim, and there is none: the ten ranks that were blocked in the reduce had already completed and published their agent calls, and the six that were starved were not blocked in any receive at $t = 0.09$ s, they were blocked in `comm.agent`. The dependency runs strictly one way. Rank 0 waited on rank 8; rank 8 waited on ranks 9, 10 and 12; ranks 9, 10 and 12 waited on worker processes that did not exist. Every receive timeout in the run is a consequence of the tail of that chain, and the binomial reduce tree is what propagated six unprovisioned ranks into sixteen failed ones.

7.4 A failure observed through AgentMPI, not a failure of it

Argued from the evidence, the protocol's own machinery detected and reported this condition correctly at every layer, and the run's illegibility budget was never spent.

Nothing hung. Every one of the sixteen ranks terminated at a bounded, configured, attributable deadline: ten at 1,800 s after entering a collective, six at 5,400 s of broker wait followed by a further 1,800 s. There is no wait in this trace whose duration is not the value of a named timeout.

Nothing was mis-classified. All sixteen `rank.error` records carry the class `ERR_TIMEOUT` and the message `receive timed out`, which is what happened; none reports a context overflow, a contract violation or an internal error. `contract_violations` is 0 and `n_retries` is 0 across all sixteen ranks — the ten term sheets that did land were contract-clean on the first attempt, so the `TermSheet` contract never had to fire and did not falsely fire either.

Nothing was silently absorbed. `broker.expire` names the aid and the exact wait; `harness.degraded` names the label of the work that was lost and the class of the loss; `job.finish` records `ok: false` and enumerates all sixteen failed ranks rather than reporting a partial success. The run cost $0.0726 and produced no output, and both facts are in the record.

And the degradation path itself worked. `safe_agent` caught the `AmpiTimeout`, contributed the identity element, emitted the trace event and kept the rank in the group — exactly the discipline its docstring argues for — and five of the six degraded ranks went on to send their empty contribution into the reduce. A protocol that fails this loudly and this legibly is behaving well.

7.5 The one genuine defect: an inverted timeout hierarchy

There is nevertheless something here I would call a real defect rather than expected behaviour, and it is a composition defect rather than a bug in any one component. The broker's expiry is 5,400 s, set by `make_executor_factory(timeout=5400.0)` in `experiments/common.py`. The collectives' receive timeout is 1,800 s, the default in `algorithms.py`. Each value is individually defensible. Composed, they are in the wrong order, and the consequence is that `safe_agent`'s degrade-and-stay-in-the-group discipline is unreachable by construction: a rank cannot rejoin a collective at $t = 5{,}400$ s that its peers abandoned at $t = 1{,}800$ s. The trace contains the proof rather than the hypothesis — ranks 9, 10 and 15 did send their degraded contributions, and those three sends are the run's three unmatched messages, delivered into the mailboxes of ranks that had been dead for 3,412 s.

The fix requires no new protocol machinery and no change to the harness's structure: the agent-call timeout must be shorter than the receive timeout of the next collective the rank will enter. Had the broker expired at, say, 900 s, the six starved ranks would have degraded at $t \approx 900$ s, rank 8 would have folded and forwarded, the root would have completed the reduce, and the run would have translated ten of sixteen sections with a ten-rank glossary and reported the six absentees honestly through `stats['degraded']` and the `PROCEED` barrier's `absent` list. A degraded book is what this harness was designed to produce under exactly these conditions, and the only thing standing between the design and the outcome was the ordering of two timeout constants.

This defect and the "observed through, not of" reading above are in tension — the shortage was not the protocol's fault, yet the protocol's own recovery path was made unreachable by the configuration around it — and §7.7 resolves which of the two owns the ordering.

7.6 What the harness should have done, and why it did not

The protocol offers `ft.revoke`, `ft.shrink` and `ft.agree` for this situation, and the trace is clear both that they would have helped and that nothing attempted them. `revoke` exists to free ranks already blocked inside a collective that can never complete, and `comm.recv` calls `_check_live()` inside its poll loop, so a single revoke would have collapsed all sixteen of the 1,800 s waits into seconds. `agree` plus `shrink` would then have rebuilt the communicator over the survivors.

Three pieces of evidence explain why none of this happened. First, `experiments/translation.py` never imports `revoke`, `shrink` or `agree`; its only fault-tolerance affordance is the `PROCEED` barrier policy at phase 5, which sits downstream of the phase-2 glossary `allreduce` and was therefore never reached. Second, the `failures` table in the fabric has zero rows and the log has zero `ft.declare_failed` events: no rank was ever declared failed, which is the precondition for `shrink`'s agreement step. Third, `role_counts` records 28 trouble events and has no recovery entry at all, and `n_recovery` is 0, so the violet recovery colour of the viewer's palette appears nowhere in this run — not because a recovery was attempted and failed, but because none was attempted.

One correction to the obvious prescription: the harness should have shrunk to *ten* ranks, not twelve. The twelve figure is the count of ranks that emitted a `coll.reduce` record, and it is a different set from the set of ranks that could do work. The twelve emitters are the seven served non-root ranks (1–7) plus the five starved ranks that folded after degrading (9, 10, 11, 13, 15); the four absentees are the root and the interior nodes whose children were starved (0, 8, 12, 14). The set of ranks that actually had an executor is $\{0..8, 14\}$, which is ten. A shrink at $t \approx 190$ s over those ten is what a correct supervisor would have produced. Whether the resulting run would have been fast is an

inference rather than a measurement, but a bounded one: `real-tr-p8-full` completed all three agent phases over eight units in 166.8 s, so a ten-rank continuation was minutes of work, not hours.

7.7 Whose defect is the ordering? A recommendation

Two of the readings above sit in apparent tension and are worth reconciling explicitly, because this run is the only evidence there is for the question. The capacity shortage was not AgentMPI's fault — six rank roles had no executor, and no protocol can conjure one. Yet the protocol's own recovery affordances were made unreachable by a timeout ordering that came from the configuration surrounding it. If the second is a real defect, whose is it?

The tension dissolves once you ask not who chose the numbers but who was in a position to check them. Nobody chose the pair. The 5,400 s comes from a default argument in the executor factory in `common.py`; the 1,800 s comes from a module-level default in `algorithms.py`; and `translation.py` passes neither, mentions neither, and contains no line that is wrong. The hazard was produced by two independently authored defaults composing across a module boundary, and there is no single site at which a careful author would have seen both. A rule that can be violated without anyone writing an incorrect line is not a rule an author can be held responsible for, and it is precisely the class of hazard this protocol already takes on itself elsewhere: rendezvous transport refuses to admit tokens that would overflow a context rather than trusting harness authors to count them, and `UNION` is offered over a semantic merge so that exactness is a property of the operator rather than of discipline. My recommendation is that AgentMPI should own this one too.

It should not, however, *forbid* the ordering, for three reasons the trace and the API make plain. First, the relation is often not knowable at `init`: both timeouts are per-call keyword arguments, a harness may legitimately vary either by phase, and a static check can only ever see the defaults. Second, the ordering is not always a defect — a harness that makes an agent call and then enters no collective has no peer waiting on it, and a long agent timeout is correct there, so refusal would break correct programs. Third, converting a latent quality problem into a startup failure is worst for exactly the runs that most need to make partial progress.

What I would recommend instead, in ascending order of value:

Warn on the defaults at `init`. The runtime already holds the executor's `default_timeout` and can see the collective defaults; when the former exceeds the latter it can emit one trace event naming both values. It is a comparison of two constants and it is cheap, and its worth is that it lands *in the log*, so a sweep over the archive could find every run carrying the hazard latently rather than only the one where it fired.

Report the ordering at the moment it becomes provable. This is stronger, needs no static reasoning about configuration, and I would rank it the highest-value addition of the three. A send that lands in the mailbox of a rank already in state `failed` is proof that a degrade-and-rejoin arrived too late, and the fabric can see that locally and mechanically at delivery time. A distinct event — `msg.orphaned`, say — would have promoted this run's single most diagnostic fact to a first-class observation instead of something an analyst had to recover by differencing 27 `msg.send` against 24 `msg.recv` and then reconstructing the reduce tree by hand. It generalises past timeouts, too: *any* degraded contribution can arrive after its group has moved on, whatever the constants, and only the fabric is positioned to notice.

Make the correct ordering the default rather than something to remember. The deeper reason the harness could not protect itself is that there is no way to ask the question "how long may I wait

here before my peers give up on me?" Threading the enclosing collective's receive timeout through as `comm.agent`'s default timeout, or exposing it as an accessor, would make the discipline that `safe_agent`'s docstring preaches into something the API supplies. Notably, `safe_agent` itself would need no edit: it is already correct, and was already correct in this run.

What should remain the harness author's responsibility is the mitigation, not the trigger. Whether a ten-of-sixteen book is worth producing is a domain judgement the protocol cannot make, so calling `revoke`, `agree` and `shrink` is properly the author's job — and §7.6 is a criticism of this harness, not of the protocol. But the author has to be *able* to make that judgement, and in this run nobody ever was: the `failures` table has zero rows, no rank was declared failed, and by the time the degrade path fired the group had been gone for 3,412 s. The mitigation primitives exist and the paper's fault table shows them working; what is missing is the trigger, and the trigger is the part only the runtime can see. That is the line I would draw.

One thing I would explicitly *not* recommend: lengthening the collective timeout to match the broker's. It repairs the ordering and is exactly backwards — it would have turned a two-hour failure into a four-hour one, and it inverts the design intent, which is that a collective should fail fast and leave the decision to a supervisor.

7.8 The incomplete reduce is not a cost-model disagreement

The generated findings flag the glossary reduce as *[warning]*: reached by 12 of 16 ranks, 12 messages logged where the closed form predicts 15. This is expected for this configuration and specifically is *not* a model failure, and the analysis tooling is right to withhold the comparison — the collective table shows – rather than ×, `model_checks_total` is 1 rather than 2, and Figure 5 omits the reduce from the plot entirely.

The reason is structural. The closed form for binomial reduce is derived for a communicator of size p: every rank contributes, every interior node folds, and $p - 1$ messages cross the tree. A run in which four interior nodes never folded logs fewer messages not because the algorithm is cheaper but because it did not happen. Comparing 12 against 15 would score the model as wrong on the evidence that the run was incomplete, which manufactures a defect out of an absence. `analysis.py`'s `Invocation.complete` gates `messages_agree` on `n_participants >= size` for exactly this reason, and its docstring names this run.

The positive form of the same point is worth stating: the one collective in this run that *did* complete, the binomial `scatterv`, logged 15 messages against a predicted 15 and 4 rounds against a predicted 4, with 0.06 s of skew. The cost model was validated by this run, on the only invocation where validation was meaningful. The reduce's skew of 5,284.58 s — first fold at 115.58 s, last at 5,400.16 s — is the honest summary of what went wrong, and it is a far more useful number than a spurious model disagreement would have been.

7.9 What this means for the paper's scaling claims

This is the most consequential thing in the run, and it is not about the run. As of commit `d4bca31`, `scripts/make_tables.py` selected the strong-scaling series by campaign label and configuration only:

```
    if c.get("glossary") and c.get("halo") and c.get("glossary_op") == "union"
        and c.get("allreduce_alg") == "reduce_bcast":  scal[c["ranks"]] = d
```

There was no test on `d["job"]["ok"]` anywhere in the translation table builder, although the same file already applies exactly that filter to microbenchmark rows in `table_model_validation`. The p=16 result matches the selector, so it was included, and the generated strong-scaling table under `paper/generated/` at that commit reads:

```
16 & 7,200 & 0.08 & 0.01 & 13.250 & 10 & 13,925 & 0.073 & 1.000
```

Every derived cell in that row is meaningless and two are actively misleading. A Karp–Flatt metric of 13.250 is a serial-fraction estimate, which cannot exceed 1 by construction. A consistency of 1.000 is what `score_translation` returns when `n_shared_entities` is 0, so a run that produced no translated text at all reported perfect terminological agreement, in the column a reader scans for quality. The 7,200 s wall figure is a timeout sum, not a duration.

The contamination propagated into the prose. The paper's §Q3 asserts that "the *contention* coefficient σ is substantial at 0.22, and the *coherency* coefficient κ is zero", and that "the Karp–Flatt metric agrees: it does not grow with p". Both statements are contradicted by the table they cite. Karp–Flatt in that table runs 0.000, 0.422, 0.380, 0.188 and then 13.250 — a seventyfold jump. And refitting `agentmpi.cost.fit_usl` on the five speedups derived from the recorded wall times gives $\sigma = 0.02$, $\kappa = 0.0465$, $R^2 = 0.404$ with p=16 included, against $\sigma = 0.220$, $\kappa = 0.0000$, $R^2 = 0.873$ without it. The 0.22 and the zero that the prose quotes are the p=1–8 fit. Because `make_tables.py` suppresses the fit below $R^2 = 0.5$, the `\NUslFit` macro at `d4bca31` in fact expanded to "not supported by single-trial data ($R^2 = 0.404$)" — so the paper simultaneously asserted a fit and printed a refusal to report one, two sentences apart, because of this one run. Its central claim about κ, the coherency ceiling that distinguishes collective coordination from group chat, rested on a number the generated macro no longer contained.

This has since been addressed. Commit `c2d4baa` partitions the series on `d["job"]["ok"]`, reports the failed attempt as a footnoted row with no derived quantities, and restores `\NUslFit` to $\sigma = 0.220$, $\kappa = 0.0000$, $R^2 = 0.873$. That is the right shape of fix: it neither launders the failure nor lets it masquerade as a measurement.

A second reason the p=16 point is unsafe, independent of the `ok` flag. The p=16 run did not translate the same book as its siblings. Its config records `units: 16` against `units: 8` for p=1, 2, 4 and 8, and because `common.split_units` advances a cursor through the text taking `n_units` chunks of at most `words_per_unit` words, total work is proportional to the unit count. The recorded `source` blocks confirm it: 10,438 words and 312 paragraphs at p=16, against 5,109 words and 149 paragraphs at every other point — 2.04× the work. (The paper quotes the nominal budget instead, 9,600 against 4,800 words for a ratio of exactly 2.00. Both are right: `split_units` appends whole paragraphs until the 600-word budget is reached, so each unit overshoots slightly and the recorded totals run 6–9% above nominal. The figures here are the ones the result artefact holds.) The table was captioned "Strong scaling: identical total work, decreasing population", and for this row that caption was false; the row's surviving wall, calls, tokens and USD cells are not comparable with the rows above them even after the derived columns were withdrawn.

This is not a driving mistake so much as a structural limit of the sweep, and that makes it worth stating rather than merely fixing. At $p = 16$ with 8 units, `n_local = max(1, 8 // 16) = 1`, so ranks 0–7 receive one unit each and ranks 8–15 receive empty lists; the halo phase then indexes `mine[0]` and raises. The strong-scaling series *cannot* be extended past $p = 8$ at this unit granularity without changing the problem. The honest reading is that the strong-scaling curve for this experiment has four points, p=1 through p=8, and that p=16 is a weak-scaling attempt that failed for reasons unrelated to either kind of scaling.

Both of these are now recorded in the paper. Commit `17aee9ce` scopes the caption's "identical total work" claim to the four completing points, states the workload difference and its $\max(1, \lfloor u/p \rfloor)$ mechanism in the prose, and rewrites the footnote so that the row's wall time is attributed to two composed deadlines rather than to a job budget. The p=16 row is now reported without a single derived quantity, and every cell that survives is marked as incommensurable. That is the correct disposition: the failure is a real result about the operating envelope and belongs in the table, but not one number of it belongs on the curve.

7.10 What the run contributes relative to its siblings

Read against `real-tr-p1/p2/p4/p8-full`, this run's distinctive contribution is a clean separation of two things the successful runs conflate. In all four successful runs `max_claim_wait_s` is 0.0: agents were available the instant they were asked for, so those runs measure the harness and tell us nothing about what happens when they are not. The p=8 run reached `max_busy` 8 of 8 and so cannot distinguish a fleet of eight from a fleet of eighty. This run reached `max_busy` 8 of 16 and a claim wait of 5,400 s, and is the archive's only observation of the protocol under executor scarcity.

That observation is worth having, and its content is narrower than the headline. The per-call service times measured here (21.6–37.5 s) sit close to p=8's (19.5–29.5 s) on byte-identical prompts: prompt token counts for `terms:u0` through `terms:u7` are identical between the two runs, and output token counts match for seven of the eight (`terms:u2` returned 248 tokens here against 255 at p=8). The whole of the apparent latency regression, `agent_latency_p50` of 126.57 s here against 46.0 s at p=8, is queueing, not slower agents. What this run establishes about scaling is therefore not that coordination cost explodes at $p = 16$ — the trace has no evidence about that, since the run never completed a collective at $p = 16$ — but that the harness's operating envelope was bounded by executor supply rather than by protocol overhead, and that the boundary is unforgiving: sixteen roles served by ten workers produced not a 10/16 slowdown but total failure, because a binomial reduce has no partial answer.

8 Threats to this reading

The provisioning diagnosis is inferential at one step. I can show from four independent fabric records that no worker process ever registered against ranks 9–13 and 15. I cannot show *why* from the trace, and the trace is the only evidence I have. A launcher that started ten workers and exited, a quota or rate limit that refused the remaining six, six worker processes that crashed before their first `RankRuntime.register`, or an operator who launched ten by hand are all consistent with what is recorded. The claim I am confident in is that the six roles had no executor; the claim "the fleet was capped at ten" is a reading of the $t = 91/t = 156$/never pattern, not a measurement. If a driver log exists outside the fabric it would settle this, and my reading of §7.6 would be unaffected either way, since the harness's obligation to shrink does not depend on why the ranks were absent.

The coordination share is a floor, and on this run an extremely loose one — but it now says so. A rank records a `coll.*` event only on *completing* its part, so every rank that blocked and then timed out contributes nothing to any coordination measure. In this run that is all sixteen of them. The generated headline therefore reads *coordination share 0.0%* for a run in which coordination was the only thing that happened, and the numerator behind it is 6.355 rank-seconds: 0.584 s from the scatter and 5.771 s from the twelve reduce folds that completed. The honest figure has to come from the log. Every rank died exactly $1{,}800.0 \pm 0.1$ s after posting its

glossary receive, so $16 \times 1800 = 28{,}800$ rank-seconds were spent blocked inside a collective, against 310.5 rank-seconds of work — 46% of the 62,570 rank-seconds the ranks actually existed for, or 25.0% against the tooling's denominator of `world_size` × wall = 115,203. Either way the recorded figure captures 0.022% of the blocking that occurred, understating it by a factor of about 4,500.

This is now disclosed rather than merely wrong, and the disclosure is the right shape. `metrics.json` carries `coordination_is_underreported: true` — set whenever a run has incomplete collectives or `ok: false` — and the generated findings list carries the matching *[warning]*, which is the fifth flagged finding above and is expected behaviour rather than a defect: the measure is correct as defined, the definition is not the quantity a reader of a degraded run wants, and saying so beats both silence and a guess. The metric itself was also rebased onto rank-seconds blocked over rank-seconds available, which is a genuine proportion; the separate *coordination in flight* row, 0.1%, is the union of blocking intervals over wall clock and answers a different and also-floored question. What no artefact can yet compute is the 28,800, because it exists only as the absence of sixteen events, and until it can the flag plus a log-derived number is the best a reader can be given.

Two other headline metrics still read against the grain of the run. "Load imbalance 1.21×" reads as near-perfect balance in a run whose defining feature is that six of sixteen ranks did nothing; `Analysis.imbalance` filters to `busy_s > 0`, so it describes only the ten served ranks. Computed over all sixteen ($37.497/(310.498/16)$) it is 1.93×, and the caveat is still needed because the filter is still in place. And "rank reattachments 20" is flagged *[ok]* and is genuinely fine, but it counts only ranks that had workers; the six that did not are invisible in it.

The agent-latency figures are not model latencies. `BrokerExecutor` computes `latency_s = finished_at - created_at`, so `agent.call` latency includes broker queue time. Every latency in this run is therefore inflated by 91 s or 156 s of waiting, and `metrics.json`'s `calibration.alpha_p50 = 125.055` is a queue-plus-service time reported as an agent invocation latency, which is not what α means in the cost model. This does not reach the paper — `table_translation` selects the calibration row by `max(agent_calls)`, which is the 31-call p=8 semantic run, not this 10-call one — but it is a near miss, and any future selector that preferred the largest p would import a contaminated α.

Two counters named `tokens_deferred` disagree by 3.1×. The `job.finish` payload reports 50,544; `metrics.json` reports 16,082. Both are correct under their own definitions — 16,082 is the token count of the three rendezvous sends, and $16{,}082 + 34{,}462 = 50{,}544$ adds every non-admitted receive including eager ones — but they share a key name across two artefacts a reader will put side by side. Neither propagates to the paper, which quotes `tokens_in + tokens_out`.

This is n=1 on the failure itself, and it is the archive's only failure. The protocol behaviours I have credited — bounded waits, correct error classes, a working degrade path, an incomplete-collective exclusion that fires when it should — are each observed exactly once here. That is enough to establish that they work in this instance and not enough to establish that they always do. Conversely, the inverted timeout hierarchy of §7.6 is a property of two constants rather than of this trace, so it does not need replication; it can be read off `common.py` and `algorithms.py` directly, and every run using both defaults has it latent.

The '7,200 s job cap' reading is a trap this run sets for every reader. It is a very natural misreading, because `ampi.launch`'s own default timeout is exactly 7,200 s, and an earlier version of the scaling footnote made it. It is wrong: `job_timeout` was 14,400 s, all sixteen errors are receive timeouts, and $7{,}200 = 5{,}400 + 1{,}800$ to within 0.06 s. The distinction changes the remedy, which is why it is worth restating even now that the footnote has been corrected. "The job cap fired" suggests a longer budget might have let the run finish; "the broker expired and then the reduce

receive timed out 1,800 s later" says the run had two hours of unused budget and a longer one would have produced an identical failure. Anyone re-deriving this run's duration from the config alone will reach the wrong conclusion, because the two numbers that produce it live in two other files.

Finally, a caution about the argument in §7.8. I have defended the exclusion of the incomplete reduce from model checking, and I believe the defence is right, but it should be recognised as a policy that cannot be validated by the run it protects. The exclusion means a genuine cost-model defect occurring in a collective that also happened not to complete would be invisible. Nothing in this trace suggests that is happening: a binomial reduce has every non-root rank send exactly once, so the complete form is 15 sends, and the 12 logged are 15 minus precisely the three that ranks 8, 12 and 14 never made, which is what the recovered topology requires. But the check that would catch it is the one being withheld, and that is worth saying out loud rather than resting on the tooling's judgement.